

VIZINHANÇA E BUSCA LOCAL

DCE770 - Heurísticas e Metaheurísticas

Atualizado em: 4 de setembro de 2025

Iago Carvalho

Departamento de Ciência da Computação



Anteriormente nós vimos o conceito de heurísticas construtivas

- No geral, estas eram baseadas em algoritmos gulosos

Estas são ferramentas indispensáveis para gerar soluções de forma rápida e *relativamente* eficiente

Entretanto, a visão míope deste tipo de algoritmo normalmente leva a soluções sub-ótimas que podem ser melhoradas (e muito!)

Soluções geradas por heurísticas construtivas podem ser refinadas utilizando uma série de mecanismos

Uma maneira interessante é buscar soluções que estão "próximas" da solução gerada

- Também chamadas se *soluções vizinhas*

É possível iterar sobre estas soluções vizinhas para caminhar no espaço de buscas Ω

- Passear tanto sobre soluções válidas quanto inválidas

PRINCÍPIOS DE BUSCAS LOCAL

Algoritmos que iteram sobre soluções vizinhas são chamados de algoritmos de busca local

- Ao invés de construirmos soluções do zero, vamos fazer buscas ao *redor* de uma solução prévia
- Esta busca é realizada iterando sobre um ou mais esquemas de vizinhança
- A busca faz uma amostragem guiada do espaço de soluções

Buscas locais tem como objetivo melhorar a solução atual

O sucesso de um algoritmo de busca local é fortemente atrelado ao esquema de vizinhança que ele utiliza

Para construirmos uma vizinhança, temos que ter em mente os mesmos conceitos utilizados em heurísticas construtivas

- Conjunto candidato
- Espaço de soluções
- Função de solução
- Função objetivo
- Função de viabilidade
 - Mecanismos para garantir a viabilidade, como visto na aula anterior

Além disso, também precisamos de uma **estrutura de vizinhança** e de **operadores de vizinhança**

- Um algoritmo pode utilizar múltiplas estruturas e operadores de vizinhança

ESTRUTURA DE VIZINHANÇA

Seja S o conjunto de soluções possíveis em Ω . Além disso, seja $s \in S$ uma solução possível

Uma estrutura de vizinhança $N(s)$ pode ser aplicado sobre soluções $s \in S$

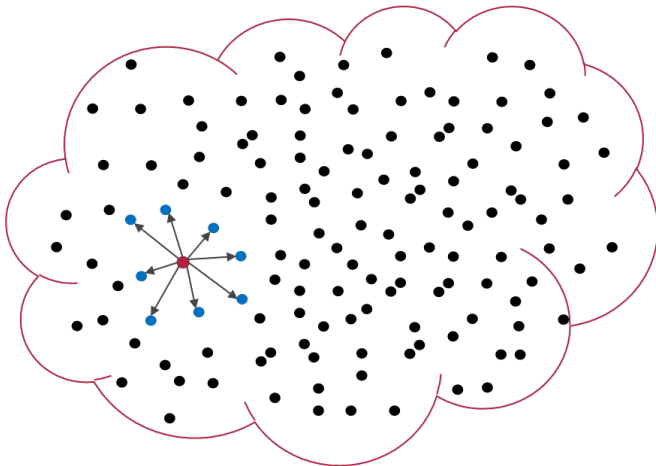
- $N(s) \subseteq S$ representa um subconjunto de soluções próximas à s em Ω

Uma solução s' é dita ser vizinha de s se e somente se $s' \in N(s)$

VIZINHANÇA - DEFINIÇÃO FORMAL

Em outras palavras, uma vizinhança é uma função $N : \Omega \mapsto 2^\Omega$

- Mapeia uma solução $s \in S$ a um subconjunto $N(s) \subseteq S$



OPERADORES DE VIZINHANÇA

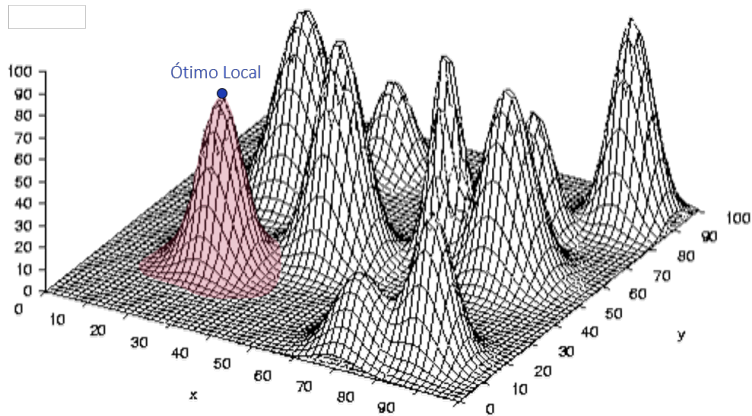
Um *operador de vizinhança* representa um **movimento** em Ω

- Uma forma de transicionar de s para s' utilizando a função de vizinhança $N(s)$

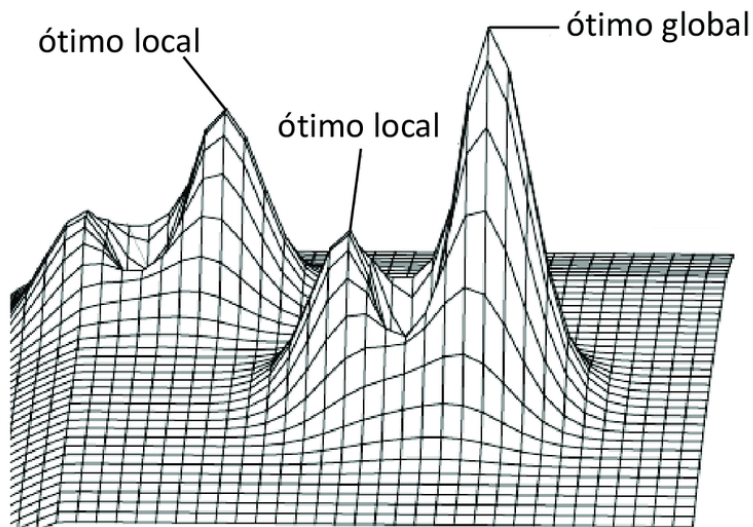
Este pode ser considerado um algoritmo que implementa a estrutura de vizinhança previamente definida

A aplicação repetida de um operador de vizinhança leva a solução até um ótimo local

ÓTIMO LOCAL



ÓTIMO LOCAL E GLOBAL



HEURÍSTICAS DE BUSCA LOCAL

Duas soluções s_1 e s_2 de S podem ou não ser vizinhas

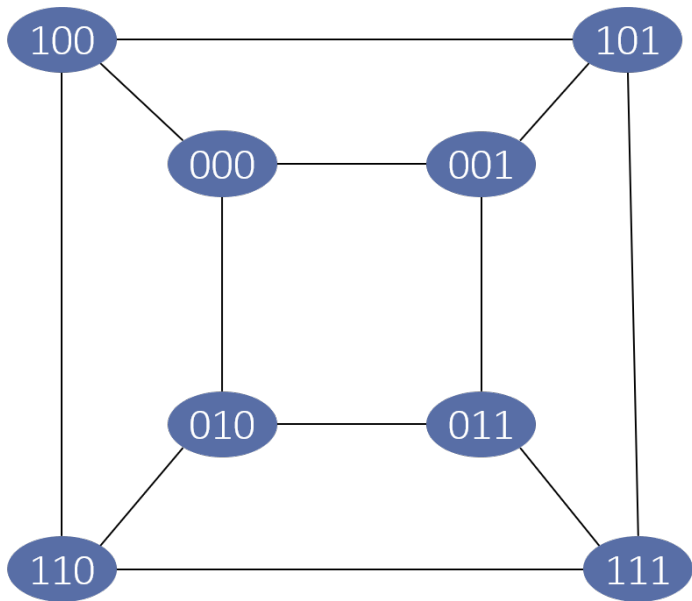
- Depende da estrutura de vizinhança definida

Heurísticas de busca local são algoritmos que possuem uma única solução

- Realizam buscas em Ω aplicando uma ou mais operadores de vizinhança sobre uma solução inicial
 - Solução inicial muitas vezes criada com uma heurística construtiva

ESQUEMAS DE VIZINHANÇA CLÁSSICOS

VIZINHANÇA NO ESPAÇO $\mathbb{B}^3$



VIZINHANÇA PARA O PROBLEMA DO CAIXEIRO VIAJANTE

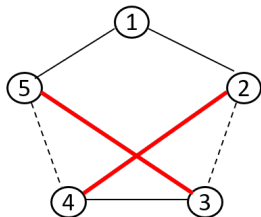
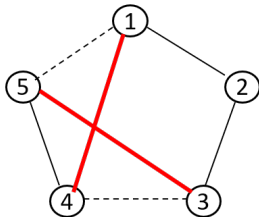
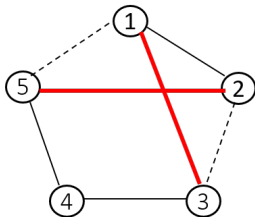
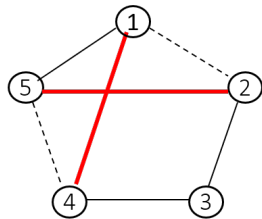
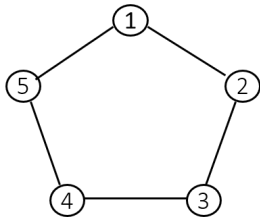
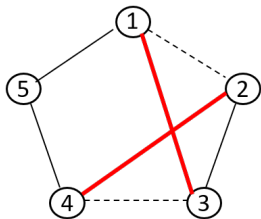
Talvez o esquema de vizinhança mais tradicional para o caixeiro viajante seja o 2-opt

1. Selecione duas arestas não adjacentes na rota atual do caixeiro
2. Remova estas duas arestas, gerando assim dois caminhos disjuntos
3. Reconecte os caminhos de uma maneira diferente

Este esquema de vizinhança é capaz de *descruzar* arestas quando o problema é representado sob um plano euclidiano

Sua complexidade é de $\mathcal{O}(n^2)$

VIZINHANÇA PARA O PROBLEMA DO CAIXEIRO VIAJANTE



VIZINHANÇA PARA O ROTEAMENTO DE VEÍCULOS

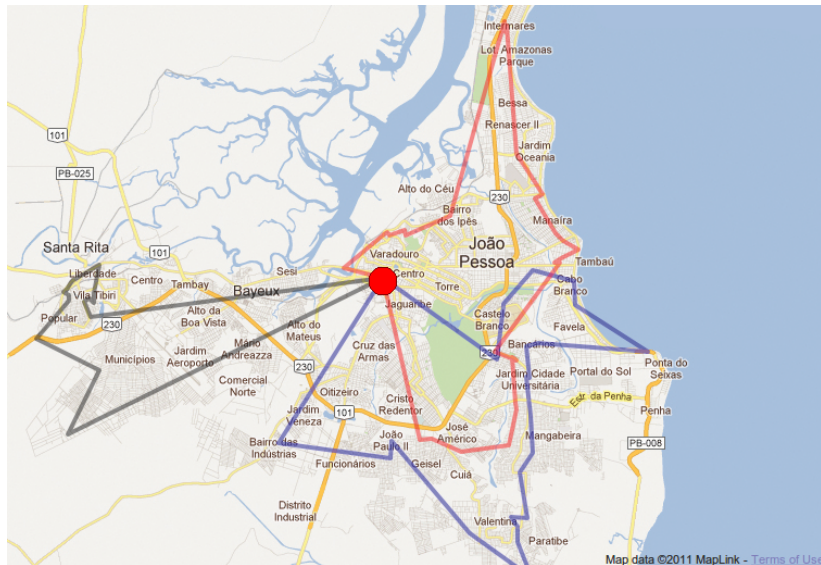
O problema de roteamento de veículos é o equivalente a um problema do caixeiro viajante com múltiplos caixeiros

Existe um depósito (cidade central) e um conjunto de clientes (cidades), além de múltiplos caixeiros (veículos)

Deve-se encontrar um conjunto de rotas de menor custo para todos os veículos

- Rotas começam e finalizam no depósito
- Cada cliente é atendido por um único veículo
- Todos os clientes são atendidos
- Minimizar o somatório dos custos das rotas

VIZINHANÇA PARA O ROTEAMENTO DE VEÍCULOS

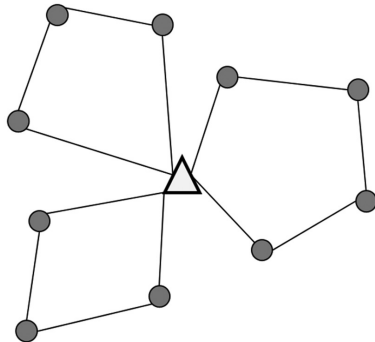
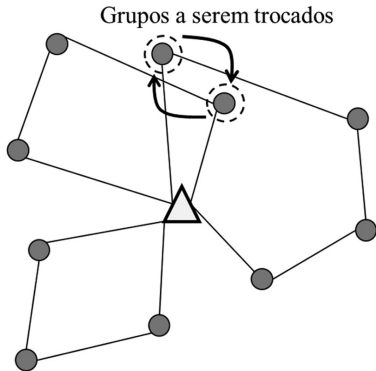


VIZINHANÇA PARA O ROTEAMENTO DE VEÍCULOS

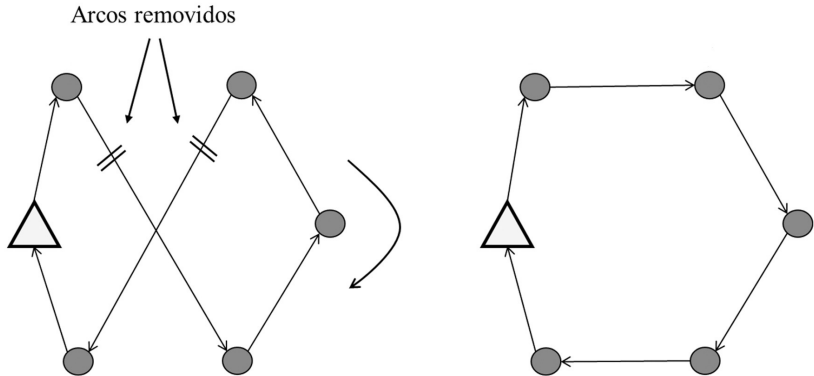
Existem diversas estruturas de vizinhança para o problema do roteamento de veículos

- 2-opt
 - Igual ao do caixeiro viajante
 - Altera a rota de um veículo
- *swap(1,1)*
 - Troca clientes entre dois diferentes veículos
- *shift*
- *or-opt*
- ...

VIZINHANÇA PARA O ROTEAMENTO DE VEÍCULOS - SWAP



VIZINHANÇA PARA O ROTEAMENTO DE VEÍCULOS - 2-OPT



ALGORITMOS DE BUSCA LOCAL

UMA PRIMEIRA HEURÍSTICA DE BUSCA LOCAL

O algoritmo de *Hill Climbing* é uma estratégia clássica para implementar algoritmos de busca local

Uma solução inicial é gerada através de uma heurística construtiva

Logo após, move-se para uma solução vizinha *aprimorante*

O algoritmo é executado até que um ótimo local seja atingido

- Não existe nenhum vizinho aprimorante

Algoritmo 1: Hill climbing

Entrada: $N(s)$ // Estrutura de vizinhança

Entrada: s // Solução inicial

1 **enquanto** s não esteja em um ótimo local **faça**

2 $\delta \leftarrow s' | s' \in N(s)$ // δ obtém soluções vizinhas de s

3 **enquanto** $\delta \neq \emptyset$ **faça**

4 $s' \leftarrow \delta_i$ // Obtem e remove um elemento de δ

5 $s \leftarrow \text{melhor}(s, s')$ // Atualiza a melhor solução

6 **retorne** s

MELHOR APRIMORANTE E PRIMEIRO APRIMORANTE

O algoritmo acima realiza a pesquisa pelo **melhor aprimorante**

- Inspecciona todas as soluções em $N(s)$
- Escolhe aquela que *mais melhora* a função objetivo

Esta opção, apesar de garantir o melhor vizinho aprimorante, pode demandar muito esforço computacional

Em alternativa, podemos utilizar a abordagem do *primeiro aprimorante*

- Assim que encontrarmos uma solução aprimorante, a busca para

Algoritmo 2: Hill climbing com primeiro aprimorante

Entrada: $N(s)$ // Estrutura de vizinhança

Entrada: s // Solução inicial

```
1 enquanto  $s$  não esteja em um ótimo local faça
2    $\delta \leftarrow s' | s' \in N(s)$  //  $\delta$  obtém soluções vizinhas de  $s$ 
3   enquanto  $\delta \neq \emptyset$  faça
4      $s' \leftarrow \delta$ ; // Obtém e remove um elemento de  $\delta$ 
5      $s'' \leftarrow \text{melhor}(s, s')$ 
6     se  $s \neq s''$  // Se achou um aprimorante
7       então
8         retorne  $s'$ 
9 retorne  $s$ 
```

Nas próximas aulas veremos alguns *frameworks* de busca local

- Meta-heurísticas

Estas são implementadas utilizando tanto heurísticas construtivas quanto operadores de vizinhança